

We want to track four specific columns for each customer: CustomerID, Name, State, and Total_Spent.

The ORC Layout: The Streamlined Stripe

In ORC, your 1,000,000 customers are split into slices of **250,000 rows** each. This means your file contains exactly **4 Stripes**.

Let us zoom into **Stripe #1** (Rows 1 to 250,000):

- **Stripe Index Data (At the top of the stripe):**
 - Tracks Min/Max for every 10,000 rows.
 - *Example:* It records that for Rows 1–10,000, the Total_Spent ranges from \$5 to \$120. For Rows 10,001–20,000, it ranges from \$80 to \$400.
- **Column Data (The heart of the stripe):**
 - All 250,000 CustomerIDs are packed together.
 - All 250,000 Names are packed together.
 - All 250,000 States are packed together.
 - All 250,000 Total_Spent values are packed together.
- **Stripe Footer (At the bottom of the stripe):**
 - Contains the exact byte offsets where the State column ends and Total_Spent begins *within this stripe*.

How a query reads ORC:

If you ask Hive for the total spent by customers in California (State = 'CA'), the execution engine reads Stripe #1 sequentially. It opens the **Stripe Index** first, realizes "CA" exists in this stripe, skips the CustomerID and Name columns entirely, and streams only the State and Total_Spent data blocks

The Parquet Layout: The Footer-First Row Group

In Parquet, those same 1,000,000 customers might be split into larger slices, say **500,000 rows** each. This gives you **2 Row Groups**.

Let us look at how **Row Group #1** (Rows 1 to 500,000) is laid out physically on disk:

- **Column Chunks (Pure Data Pages):**
 - A massive block of 500,000 CustomerIDs (split into smaller 1MB pages).

- A massive block of 500,000 Names.
- A massive block of 500,000 States.
- A massive block of 500,000 Total_Spent values.
- **Note:** There are **no indexes** and **no footers** mixed in between these columns. It is raw, continuous data.

Where is the metadata?

It is all saved at the very end of the entire file in the **Global File Footer**.

[Row Group 1 Data] -> [Row Group 2 Data] -> [GLOBAL FILE FOOTER]

|-- Row Group 1 Metadata

| |-- State Min/Max: "AK" to "WY"

| |-- Total_Spent Offset: Byte 4000 to 9000

|-- Row Group 2 Metadata

|-- ...

How a query reads Parquet:

If Spark runs that same query (`State = 'CA'`), it does not start at the beginning of the file.

1. Spark shoots straight to the **Global File Footer** at the absolute end of the file.
2. It reads the metadata for Row Group #1 and checks the State statistics. It sees that "CA" falls between "AK" and "WY".
3. The footer gives Spark the exact map: *"The State data for Row Group #1 starts at exactly Byte 2000, and Total_Spent starts at Byte 4000."*
4. Spark makes a precise vector call to jump directly to those exact byte positions, completely ignoring everything else.

The Practical Difference

- **ORC** is like a book where **each chapter has its own mini-table of contents** at the start of the chapter. If you are reading the book from front to back, it flows perfectly.

- **Parquet** is like a textbook where **the comprehensive index is at the very back of the book**. You flip to the back index first to find the exact page numbers you need, and then you jump directly to those pages

ORC is optimized for **Apache Hive** because they were designed together by the same engineering community to solve Hive's early performance bottlenecks. [1, 2]

While formats like Parquet were built to be general-purpose, ORC contains internal mechanics specifically tailored to match how Hive processes queries, manages transactions, and interacts with the **Hadoop Distributed File System (HDFS)**.

1. Zero Network Over-Seeking on HDFS

HDFS is designed to stream large blocks of data sequentially rather than making frequent random jumps (seeks) across a file. [1, 2]

- **The Problem:** In a file format where all metadata is at the end of the file, a Hive query engine running on HDFS has to make a network request to read the very end of a file block, and then issue *new* network requests to jump back to the beginning to read the actual rows.
- **The ORC Solution:** ORC places **Index Data** at the absolute beginning of every single stripe Apache ORC File Structure. As Hive streams an HDFS block sequentially from top to bottom, it hits the index first, reads it, and immediately knows whether to pull or skip the upcoming column data in that exact stripe. This keeps HDFS data streaming smoothly with minimal network lag. [1, 2]

2. Built-in Hive ACID Transactions

Hive supports fully transactional workloads (**ACID**—Atomicity, Consistency, Isolation, Durability) allowing users to run `INSERT`, `UPDATE`, and `DELETE` statements on data lakes.

ORC is the foundational format that makes this possible. [1, 2, 3, 4, 5]

- **Delta Files:** When you update a row in Hive, Hive does not rewrite the giant base ORC file. Instead, it writes a tiny **delta file** containing just the changes, formatted as an ORC file. [1]
- **ACID Columns:** ORC natively includes hidden system columns (operation, originalTransaction, bucket, rowId, currentTransaction) that Hive relies on. [1]
- **Compaction:** During a read query, Hive uses ORC's internal row-identifiers to seamlessly merge the original base file with the delta files in real time. Hive's

background "Compactor" also relies on ORC to cleanly merge these delta layers into a new base file without locking the table.

3. Vectorized Query Execution

Hive utilizes a feature called **Vectorized Query Execution**. Instead of passing data through the query engine row-by-row, Hive processes data in blocks of **1,024 rows at a time**. [1]

- ORC's internal structure perfectly aligns with this. Because ORC's row index maps data at a tight granularity of every **10,000 rows**, it easily feeds Hive's execution engine these uniform, vectorized batches of 1,024 rows. [1, 2]
- This allows the CPU to use SIMD (Single Instruction, Multiple Data) operations, processing thousands of customer records in a single CPU instruction cycle. [1, 2]

4. Direct Integration with Hive Metastore

ORC embeds Hive-native data types and schema definitions directly inside its file metadata. [1]

- When Hive reads an ORC file, it can instantly map complex Hive types (like STRUCT, MAP, and UNION) without requiring heavy translation layers.
- It also allows Hive to perform **Type Promotion** (e.g., automatically reading an older INT column as a BIGINT if the table schema changes), which prevents queries from failing due to minor schema evolutions.